Министерство образования и науки Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого
—
Институт компьютерных наук и кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА №2

«Компилятор, компоновщик и динамические библиотеки»

по дисциплине «Технологии разработки современного программного обеспечения»

Санкт-Петербург
2025

1 ЦЕЛЬ РАБОТЫ

Целью работы – изучение этапов сборки программы с учетом особенностей статической и динамической компоновки и практическое освоение средств автоматизации данного процесса.

2 ЗАДАЧИ

В рамках выполнения лабораторной работы необходимо решить следующие задачи:

1. Изучить синтаксис файла конфигурации (Makefile) системы сборки make.
2. Разработать программу, которая формирует PNG-файла с нарисованным в нем текстовым сообщением с помощью шрифта, загруженного из файла *.ttf с использованием библиотеки zlib [1], libpng [2], freetype [3].
3. Разработать Makefile для сборки трех версий программы:
   — со статической компоновкой трех библиотек;
   — с динамическая загрузка всех трех библиотек в начале работы;
   — в виде blob-файла, загружаемого с диска при помощи загрузчика elf-loader [4].

3 УСЛОВИЕ

При выполнении лабораторной работы необходимо выполнить следующие требования:

1. Программа должна компилироваться с помощью gcc и Makefile в операционной системе Linux или WSL.
2. В исходном коде программы необходимо использовать конструкции #ifdef <флаг> <исходный код> #endif для управления условной компиляцией при сборке различных версий программы.

3. Программа в независимости от типа сборки по результатам своей работы должна не только генерировать PNG-файл с нарисованным текстовым сообщением в нем, но и печатать на экран общее время своей работы и время, затраченное на загрузку системы (от начала main до конца своей работы).

4. Файл со шрифтом, имя PNG-файла и печатаемый текст должны поступать в программу из аргументов командной строки.

5. Программа не должна генерировать исключения или преждевременно завершать свою работу в независимости от аргументов командной строки.

6. Структура каталога проекта должна иметь следующий вид:

— <имя_проекта>\ – каталог с корневым Makefile;

— <имя_проекта>\zlib\ – исходные тексты библиотеки zlib и Makefile для её сборки;

— <имя_проекта>\freetype\ – исходные тексты библиотеки freetype и Makefile для её сборки;

— <имя_проекта>\libpng\ – исходные тексты библиотеки libpng и Makefile для её сборки;

— <имя_проекта>\app\ – исходные тексты программы и Makefile для её сборки;

— <имя_проекта>\build\ – каталог с артефактами сборки.

7. Конфигурационный файл Makefile должен поддерживать следующие команды:

— make static – компиляция исходников в версии со статической компоновкой;

— make dynamic – компиляция исходников в версии с динамической компоновкой;

— make blob – компиляция исходников в версии с компоновкой в виде блоба;

— make clean – удаление всех артефактов сборки файлов;

— make all – удаление старых бинарных файлов и компиляция исходников в версиях static, dynamic и blob.

8. Сборка библиотек zlib, libpng, freetype и загрузчика elf-loader должна осуществляться из исходного кода через корневой Makefile (допускается делегирование сборки стандартным Makefile).

9. Программы, собранная в виде blob-файла, должна:

— иметь формат ELF;

— не использовать стандратную библиотеку (-nostdlib);

— собираться с опциями -pie и -fPIE компилятора gcc;

— обладать одной точкой входа, которая и будет выполнять необходимые действия;

— не иметь таблиц импорта и экспорта;

— в качестве аргументов для точки входа будет передаваться структура с параметрами программы (именем файла шрифта, именем png, строкой), а так же таблицей указателей на необходимые функции (fopen, fread, fwrite,…).

10. Конфигурационный файл Makefile должен обеспечивать инкрементную сборку программы (пересборка только измененных файлов с исходным кодом).

11. Для удобной работы с программой в версиях static, dynamic и blob необходимо использовать суффиксы для имен итоговых бинарных файлов, например: app-static, app-dynamic, app-blob.

4 ПОДГОТОВКА К ВЫПОЛНЕНИЮ ЛАБОРАТОРНОЙ РАБОТЫ

Для выполнения заданий лабораторной работы необходимо:

— Linux-подобная операционная система (в качестве основной ОС либо в качестве виртуальной машины);

— утилиты сборки ПО на языках программирования Си, Си++ (build essentials) [5];

— утилита автоматизации процесса сборки ПО – make [6];

— исходные файлы библиотек zlib, libpng, freetype [1, 2, 3].

5 ЭТАПЫ ВЫПОЛНЕНИЯ

Процесс выполнения лабораторной работы включает в себя следующие этапы:

1. Первый этап (*занятие №1*):

— написание исходного кода тестовой программы:

i. main.c – файл с точкой входа;

ii. funcs.h – заголовочный файл с объявлением функций;

iii. funcs.c – файл с определением функций из funcs.h.

— разработка Makefile для сборки тестовой программы;

— проверка обеспечения инкрементной сборки программы;

— внесение при необходимости исправлений в Makefile для обеспечения инкрементной сборки программы.

— сборка статических библиотек (zlib, libpng, libfreetype) с помощью стандартных MakeFile;

— подготовка структуры директорий проекта;

— разработка программы для создания PNG-файла с заданным текстом.

2. Второй этап (*занятие №2*):

— реализация Makefile для сборки программы с целью static.

— сборка динамических библиотек (zlib, libpng, libfreetype) с помощью стандартных MakeFile;

— внесение изменений в исходный код разработанной ранее программы для обеспечения загрузки библиотек в начале её исполнения;

— добавление цели dynamic в Makefile для сборки программы.

3. Третий этап (*занятие №3*):

— разработка blob-версии программы;

— загрузка и сборка загрузчика elf-loader;

— добавление цели blob в Makefile для сборки программы.

4. Четвертый этап (*занятие №4*):

— реализация корневого Makefile, который обеспечивает сборку библиотек, загрузчика elf-loader и разработанной программы из исходного кода.

— проверка обеспечения инкрементной сборки программы.

6 ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ

6.1 Обеспечение загрузки и исполнения blob-версии разработанной программы в системе под управлением ОС Windows

Цель дополнительного задания – получение опыта реализации переносимого программного модуля.

В рамках выполнения дополнительного задания необходимо:

1. Изучить особенности загрузки и исполнения ELF-файлов в ОС Windows.
2. Провести анализ существующих загрузчиков ELF-файлов для ОС Windows, имеющих открытый исходный код.
3. Разработать загрузчик для blob-версии разработанной программы.

В отчёте необходимо привести описание этапов загрузки ELF-файла и фрагменты исходного кода разработанного загрузчика, демонстрирующие их выполнение.

6.2 Разработка Makefile для библиотек zlib, libpng, freetype

Цель дополнительного задания – получение опыта анализа стандартных Makefile.

В рамках выполнения дополнительного задания необходимо:

1. Изучить стандартных Makefile для сборки библиотек.
2. Разработать собственный Makefile, обеспечивающий сборку статической и динамической версии заданных библиотек.

7 ТРЕБОВАНИЯ К ОТЧЕТУ

Отчет должен включать следующие пункты:

1. Цель работы.
2. Задачи с указанием выполненных дополнительных заданий.
3. Ход работы – краткое описание этапов выполнения работы.
4. Выводы

8 ССЫЛКИ

1. Библиотека zlib. URL: http://www.zlib.net/
2. Библиотека libpng. URL: http://www.libpng.org/pub/png/
3. Библиотека freetype. URL: http://www.freetype.org/
4. Загрузчик elf-loader. URL: https://github.com/erincandescent/elfload
5. Пакет утилит для сборки ПО build-essential. URL: https://packages.debian.org/ru/sid/build-essential
6. Утилита Make. URL: https://wiki.debian.org/Make
7. Описание Makefile. URL: https://habr.com/ru/articles/155201/
8. Руководство новичка по эксплуатации компоновщика. URL: https://habr.com/ru/articles/150327/
9. Перенаправление функций в разделяемых ELF-библиотеках. URL: https://habr.com/ru/articles/106107/